

WEB APPLICATION SECURITY ASSESSMENT OF OWASP JUICE SHOP

Student Cybersecurity Project Report

1. Introduction

Web applications are frequently exposed to security threats because they process user input, authentication data and sensitive information. This project presents a security assessment of OWASP Juice Shop, an intentionally vulnerable web application designed for security training.

The assessment was performed in a controlled local environment using Kali Linux. The practical work documented in this report focuses on reconnaissance, HTTP request and response analysis, and identification of common web application vulnerabilities.

2. Objective

The objective of this project is to understand the process of performing a basic web application security assessment in an authorized laboratory environment.

- Network and service enumeration
- Web application mapping
- HTTP request and response analysis
- Identification of common web vulnerabilities
- Vulnerability documentation and remediation

3. Scope

Item	Details
Target	OWASP Juice Shop
Target Address	127.0.0.1
Port	3000
Environment	Local Kali Linux lab
Authorization	Intentionally vulnerable training application

4. Lab Environment

Component	Details
Operating System	Kali Linux
Target	OWASP Juice Shop
Target IP	127.0.0.1
Target Port	3000
Web Browser	Firefox / Chrome
Network Tool	Nmap
Web Testing	Burp Suite

5. Tools Used

Tool	Purpose
Nmap	Port and service enumeration
Burp Suite	HTTP request and response analysis
Web Browser	Application interaction and validation

6. Methodology

The assessment followed a basic security-testing workflow from target setup through reconnaissance, application analysis, vulnerability testing, validation, and documentation.

Stage	Activity
1	Target setup
2	Reconnaissance
3	Port and service enumeration
4	Application mapping
5	HTTP request analysis
6	Vulnerability testing
7	Validation
8	Risk assessment
9	Remediation
10	Final documentation

7. Reconnaissance

The first stage of the assessment was to determine whether the target was reachable and identify exposed services.

Command:

```
nmap -sV 127.0.0.1 -p 3000
```

```
(kali@kali)-[~]
$ nmap -sV 127.0.0.1 -p 3000
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-11 11:36 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000072s latency).

PORT      STATE SERVICE VERSION
3000/tcp   open  ppp?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
_ _ _
SF:Port3000-TCP:V=7.99%I=7%D=8/11%Time=6A7B4192%P=x86_64-pc-linux-gnu%r(Ge
SF:tRequest,2668,"HTTP/1.1\x20200\x200K\r\nAccess-Control-Allow-Origin:\x
SF:20*/\r\nX-Content-Type-Options:\x20nosniff\r\nX-Frame-Options:\x20SAMEO
SF:RIGIN\r\nFeature-Policy:\x20payment\x20'self'\r\nX-Recruiting:\x20/#/jo
SF:bs\r\nAccept-Ranges:\x20bytes\r\nCache-Control:\x20public,\x20max-age=0
SF:\r\nLast-Modified:\x20Tue,\x2011\x20Aug\x202026\x2015:31:00\x20GMT\r\nE
SF:Tag:\x20W/\x20b1-19ff172cccc"\r\nContent-Type:\x20text/html;\x20charse
SF:t=UTF-8\r\nContent-Length:\x209393\r\nVary:\x20Accept-Encoding\r\nDate:
SF:\x20Tue,\x2011\x20Aug\x202026\x2015:36:50\x20GMT\r\nConnection:\x20clos
SF:e\r\n\r\n<!--\x20\x20\x20Copyright\x20(c)\x202014-2026\x20Bjoern\x20
SF:20Kimminich\x20&\x20the\x20OWASP\x20Juice\x20Shop\x20contributors\x20\x20
SF:20\x20\x20SPDX-License-Identifier:\x20MIT\x20\x20-->\x20<!doctype\x2
SF:0html>\x20<html\x20lang=\x20en\x20data-beasties-container>\x20<head>\x20
SF:\x20<meta\x20charset=\x20utf-8\x20>\x20\x20<title>OWASP\x20Juice\x20Shop</
SF:0title>\x20<meta\x20modern\x20and\x20sophisticated\x20insecure\x20web\x20app
SF:0location>\x20\x20<meta\x20name=\x20viewport\x20content=\x20width=device
SF:0ce-width,\x20initial-scale=1\x20>\x20\x20<link\x20rel=\x20preconnect\x2
SF:0href=\x20htt" %r(Help,2F,"HTTP/1.1\x20400\x20Bad\x20Request\r\nConnecti
SF:on:\x20close\r\n\r\n")%r(NCP,2F,"HTTP/1.1\x20400\x20Bad\x20Request\r\n
SF:0connection:\x20close\r\n\r\n")%r(HTTPOptions,EA,"HTTP/1.1\x20204\x20No
SF:\x20Content\r\nAccess-Control-Allow-Origin:\x20*\r\nAccess-Control-Allow
SF:0Methods:\x20GET,HEAD,PUT,PATCH,POST,DELETE\r\nVary:\x20Access-Control
SF:l-Request-Headers\r\nContent-Length:\x20\r\nDate:\x20Tue,\x2011\x20Aug
SF:\x202026\x2015:36:50\x20GMT\r\nConnection:\x20close\r\n\r\n")%r(RTSPReq
SF:uest,EA,"HTTP/1.1\x20204\x20No\x20Content\r\nAccess-Control-Allow-Orig
SF:in:\x20*\r\nAccess-Control-Allow-Methods:\x20GET,HEAD,PUT,PATCH,POST,D
SF:ELETE\r\nVary:\x20Access-Control-Request-Headers\r\nContent-Length:\x20
SF:0\r\nDate:\x20Tue,\x2011\x20Aug\x202026\x2015:36:50\x20GMT\r\nConnectio
SF:n:\x20close\r\n\r\n")%r(RPCCheck,2F,"HTTP/1.1\x20400\x20Bad\x20Request
SF:\r\nConnection:\x20close\r\n\r\n\r\n");

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.43 seconds
```

Figure 1: Nmap service enumeration

Port 3000 was found open. Although Nmap did not automatically identify the service correctly, the returned HTTP response and application fingerprint indicated that the service was running OWASP Juice Shop.

- Port: 3000/tcp
- State: Open
- Protocol: TCP
- Service: HTTP-based web application
- Application: OWASP Juice Shop

8. Web Application Analysis

Burp Suite Proxy was used to intercept HTTP requests generated by the application. This allowed the request structure, parameters, headers and server responses to be inspected.

The captured traffic was used to understand how the application handled authentication and user-controlled input before validating the observed behavior.

Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparator
Logger
Organizer
Extensions
Discover

Site map
Scope
Issues

Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

	Host	Method	URL	Params	Status code	Length	MIME
> localhost:3000	http://localhost:3000	GET	/styles.css		304	393	
> localhost:4200	http://localhost:3000	GET	/socket.io/?EIO=4&transport=websocket...	✓	101	129	
	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&...	✓	200	230	text/css
	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&...	✓	200	215	text/css
	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&...	✓	200	230	text/css
	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&...	✓	200	215	text/css
	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&...	✓	200	1276	JavaScript
	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&...	✓	200	215	text/css
	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&...	✓	200	499	JavaScript
	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&...	✓	200	250	JavaScript
	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&...	✓	200	262	JavaScript

Request

Response

Pretty
Raw
Hex

Pretty
Raw
Hex
Render

```

1 GET /styles.css HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
  Firefox/140.0
4 Accept: text/css,*/*;q=0.1
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost:3000/
9 Cookie: language=en; welcomebanner_status=dismiss; token=
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJKYXRRhIjp7ImlkIjoynWidXNlcmShbWUu
  OiIiLCJlbnRpbGUiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAi
  DmhyYXN0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAi
  4iOiIiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAi
  1YmnpYy9pbWFnZXN0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAi
  dGJlZS60dGJlZS60dGJlZS60dGJlZS60dGJlZS60dGJlZS60dGJlZS60dGJlZS60dGJl
  iwidXBkYXRlZEF0IjoieMjYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYyYy
  FOIjpuYWxsZXN0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0
  pbWFnZXN0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAiLCJ0eXAiOiJ0eXAi
  v8pFmT5JV03hcE5C-jD9VSypk3ZfWYCKBA55eflWLSsaJ70P151dH2maER4Hx6xtWuYB5s;
    continueCode=
    wWq2LVP8MrgezNmoRk4EQ37ABXI7S88hmRcLoG5JxyvjD90KYabpZX68nlw1
10 If-Modified-Since: Tue, 11 Aug 2026 05:23:17 GMT
11 If-None-Match: W/"24f14-19fef466988"
12 Priority: u=4

```

```

1 HTTP/1.1 304 Not Modified
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Accept-Ranges: bytes
8 Cache-Control: public, max-age=
9 Last-Modified: Tue, 11 Aug 2026
10 ETag: W/"24f14-19fef466988"
11 Date: Tue, 18 Aug 2026 19:28:53
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15

```

Figure 2: Burp Suite HTTP request/response capture

9. Vulnerability Findings

The following findings were documented from the testing performed against the intentionally vulnerable local Juice Shop application.

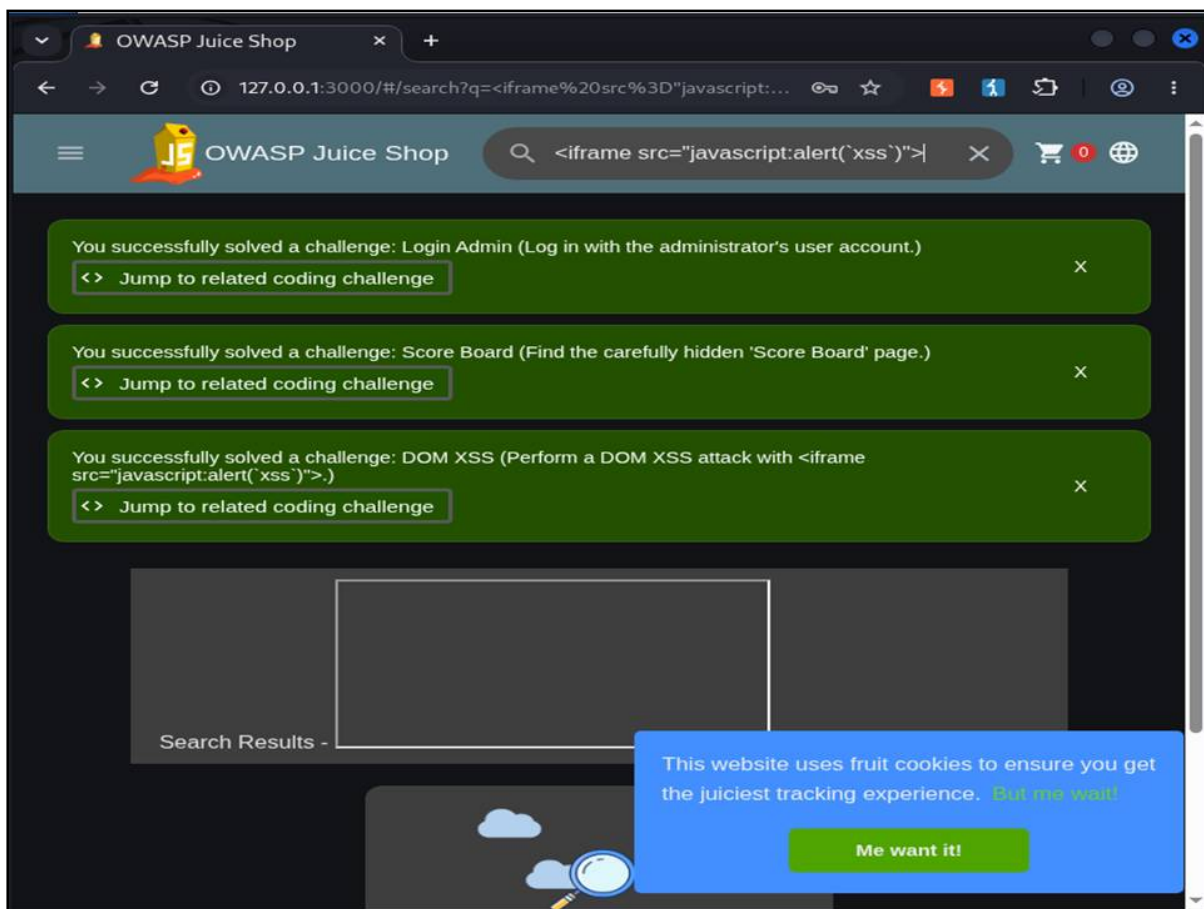


Figure 4: DOM-based XSS successful challenge/evidence

Impact: Successful client-side script execution can allow malicious actions to be performed in the context of a victim's browser, depending on where the vulnerable input is used.

Remediation:

- Avoid inserting untrusted input directly into dangerous DOM sinks.
- Prefer safe DOM APIs and context-appropriate output encoding.
- Validate user input where appropriate.
- Use a suitable Content Security Policy as an additional defense layer.

9.3. Broken Access Control

Broken Access Control was demonstrated by testing access to another user's basket/resource within the authenticated Juice Shop environment. The observed request returned the basket data with a successful HTTP response, indicating that server-side authorization did not adequately restrict access to the requested resource.

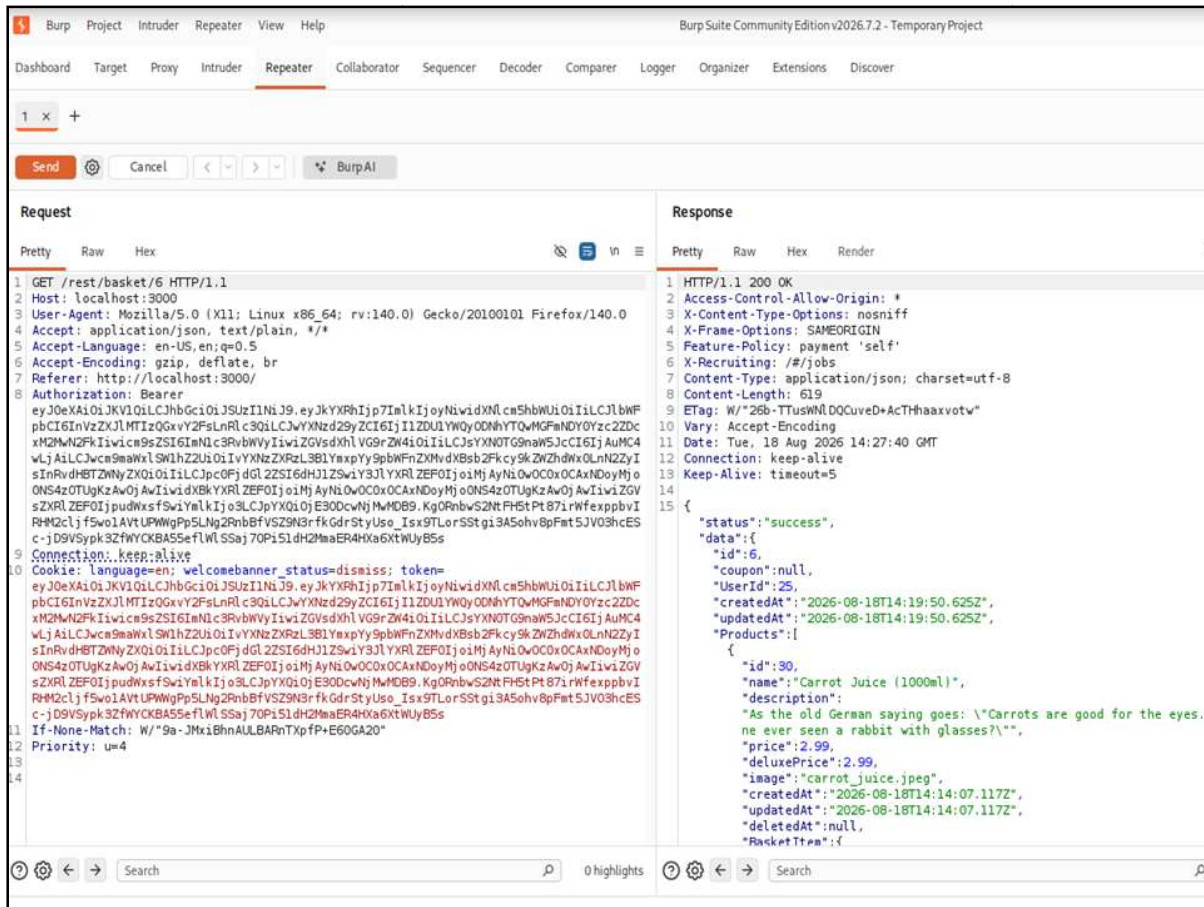


Figure 5: Broken Access Control: authenticated request and returned basket data

Impact: An access-control weakness can allow one authenticated user to access resources belonging to another user. Depending on the affected endpoint, this may expose private data or allow unauthorized modification of another user's information.

Remediation:

- Perform authorization checks on every protected server-side resource.
- Do not rely on client-side identifiers or UI restrictions for authorization.
- Verify that the authenticated user's identity is authorized to access the requested object.
- Use deny-by-default access-control policies and test object-level authorization.

10. Risk Summary

Finding	Risk	Reason
SQL Injection	High	User-controlled input can influence database query behavior.
DOM-based XSS	Medium	User-controlled input can reach a browser-side execution sink.
Broken Access Control	High	An authenticated user can access another user's resource/data.

The risk ratings above are project-level assessments based on the demonstrated behavior and should not be interpreted as a formal CVSS score.

11. Remediation

The main remediation priorities identified from this assessment are:

- Use parameterized database queries and strong server-side input validation.
- Apply safe handling of user-controlled data in browser-side DOM operations.
- Implement object-level authorization checks for every protected resource.
- Apply least privilege to application and database accounts.
- Repeat security testing after fixes to verify that the vulnerabilities have been removed.

12. Conclusion

This assessment provided a practical evaluation of the OWASP Juice Shop application in a controlled local environment. The assessment followed a structured approach beginning with reconnaissance and service enumeration, followed by HTTP traffic analysis and manual testing of application functionality.

The testing demonstrated several common web application security weaknesses, including SQL injection affecting the authentication mechanism, DOM-based cross-site scripting, and broken access control that allowed access to another user's basket data. These findings demonstrate how insufficient input handling and authorization controls can affect the confidentiality and integrity of application data.

The assessment also provided practical experience with Nmap and Burp Suite for reconnaissance, request interception, request modification, and validation of security findings. Each demonstrated vulnerability was documented with evidence and corresponding remediation recommendations.

Overall, the project helped establish a basic end-to-end understanding of web application security assessment, from identifying the exposed application surface to validating vulnerabilities and documenting their potential impact.

13. References

- OWASP Foundation, *OWASP Juice Shop – Official Documentation and Companion Guide*.
- OWASP Foundation, *OWASP Top 10: Web Application Security Risks*.
- OWASP Foundation, *Web Security Testing Guide (WSTG)*.
- PortSwigger, *Burp Suite Documentation*.
- Nmap Project, *Nmap Reference Guide*.

Prepared by Pritam Bose